ARTIFICIAL SOCIAL NETWORKS · PITCH

Closing the Transitivity Gap

A configuration model with triangle stubs for the Amazon co-purchase graph

Zoha · Amina · Arnisha · 10 November 2026

THE PROBLEM

No model gets Amazon's degrees and triangles at once

Model (Week 4 scorecard)	Max. degree	Avg. clustering	Transitivity
Amazon (real)	549	0.397	0.205
Configuration model	549	<0.001	<0.001
Chung–Lu	542	<0.001	<0.001
Holme–Kim	5647	0.517	0.032
Degree-corrected SBM	555	0.021	0.007

The configuration model has exact degrees but no triangles; Holme–Kim has triangles but wrong degrees. That matters: triangles change how epidemics and rumours spread (Miller, 2009).

RESEARCH QUESTION

To what extent can a triangle-based configuration model reproduce the clustering structure of the Amazon co-purchase network while preserving its degree sequence?

Target values: transitivity 0.205 and average clustering 0.397, with every node’s degree kept exactly.

334 863	925 872	667 129	549
nodes	edges	triangles	max. degree

OUR APPROACH

Two stub lists: pairs make edges, triples make triangles

Edge stubs: node i appears s_i times

Stub list
A A B C D D

→

Shuffle
C A D B A D

→

Every 2 = one edge
C–A · D–B · A–D

Triangle stubs: node i appears t_i times

Stub list
A B C D E F

→

Shuffle
E A C F B D

→

Every 3 = one triangle
E–A–C · F–B–D

$$k_i = s_i + 2t_i$$ each triangle gives a node two neighbours

$$t_i = \min(\Delta_i, \lfloor k_i/2 \rfloor)$$ Δ_i = real triangles of node i

OUR CONTRIBUTION

The model is Newman's. What we add is the analysis around it.

1 · Applied to Amazon, predicted in advance

Parameters read per node from com-Amazon; transitivity and its upper bound computed before building.

2 · Degree-exact implementation

Redraw and degree-preserving swaps instead of deleting; packed NumPy arrays so it can scale.

3 · The gap explained, then attacked

Clustering by degree shows where triangles are lost; a K4 extension targets exactly that part.

4 · Evaluation beyond one number

Baselines, an SIR behaviour test, and a cost analysis for 10^9 nodes with the Week 5 method.

ESTIMATE BEFORE WE BUILD

We can compute the reachable transitivity in advance

$$T_{max} \approx \frac{\Sigma\, t_i}{\tfrac{1}{2}\,\Sigma\, k_i\,(k_i - 1)}$$

$$t_i = \min(\,\Delta_i\,,\,\lfloor k_i/2 \rfloor\,)$$

Δ_i = real triangles of node i, k_i = its degree.
Denominator = connected triples, the same in model and real graph.

0.205
real transitivity of Amazon

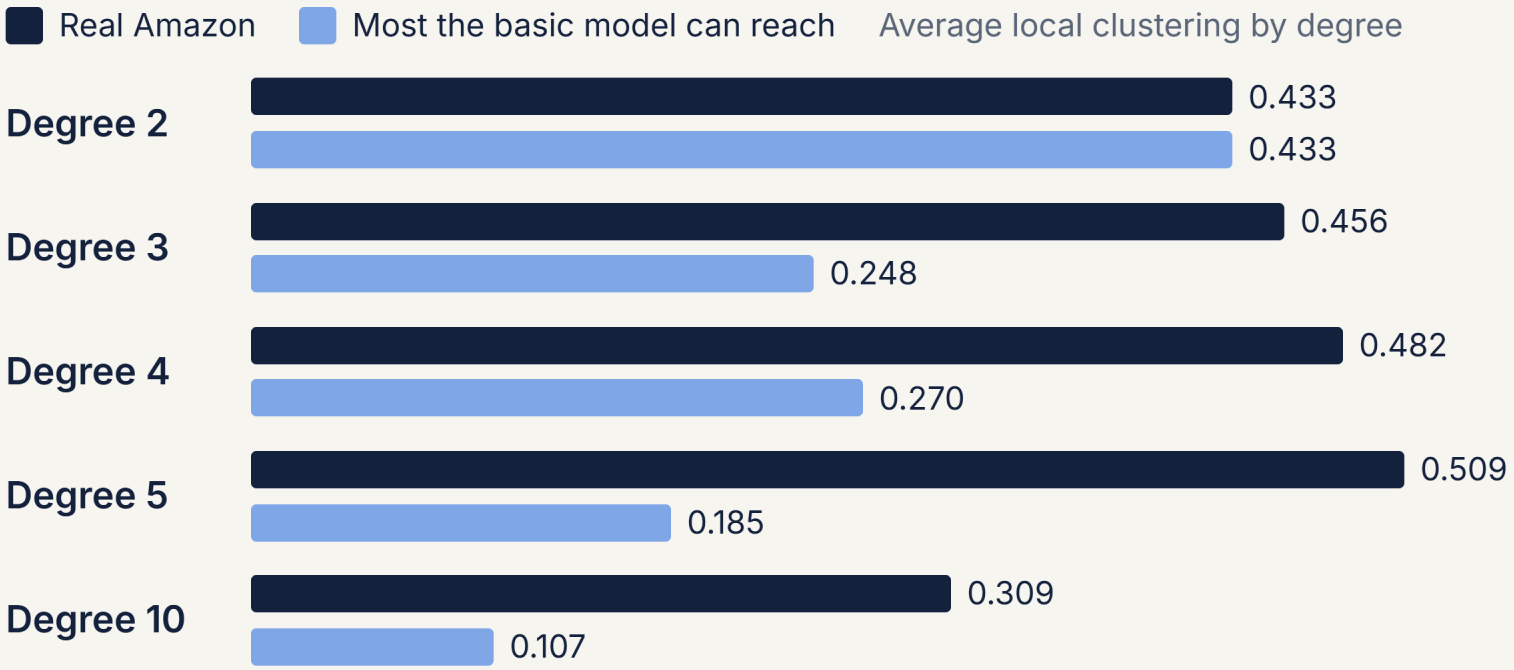
0.077
upper bound of the basic triangle-stub model

37.5 %
of triangles can be represented

56 % of Amazon's nodes sit in more triangles than ⌊k/2⌋ allows. So this basic model cannot reach 0.205, and we know it before writing the generator. The bound holds for triangles placed on purpose in this model, not for every possible generator.

EXPLAINING THE GAP IN ADVANCE

The gap sits in dense groups of low-degree products



Example: K4

Four products all bought together: each has degree 3 and sits in 3 triangles (clustering 1.0). Our model allows 1 triangle (0.33).

Next step: K4 stubs for these groups.

7 · Computed on com-Amazon before building: real local clustering vs. the cap ⌊k/2⌋ / (k(k−1)/2) per node, averaged by degree

EVALUATION

Success criteria are fixed before we build

Metric	Kind (Week 1)	Counts as success
Transitivity	Statistics	Within ±5 % of our prediction (0.077)
Avg. clustering	Statistics	Within ±5 % of our prediction (0.205)
Degree sequence	Statistics	100 % of nodes exact; redraws reported
Clustering by degree c(k)	Statistics	Matches the real curve below the cap
SIR epidemic	Behaviour	Closer to Amazon than the configuration model (≥ 50 runs)
Avg. path length	Statistics	Reported only, not a target

Baselines (Week 4): configuration model, Chung–Lu, Holme–Kim, all with the same script. Planned: ≥ 5 graphs per model with different seeds, mean ± standard deviation.

SOFTWARE STACK

Python with packed arrays, run on the university cluster

Area	Choice	Why
Language	Python 3	Known to the team, strong graph libraries
Generator	NumPy	Packed arrays; Python objects cost 10–100× more memory
Measuring	networkit	C++ core, parallel triangles and paths
Baselines, tests	NetworkX	Configuration model, Holme–Kim built in
Statistics, plots	SciPy, matplotlib	KS test, c(k) plot
Compute	SCC cluster	One node with 2.3 TB RAM fits 10^9 nodes
Code, report	git.sbg.ac.at	University GitLab, as required

COST ANALYSIS

1 billion nodes: about 100 GB, 15 minutes, €11 700

Assumption: $n = 10^9$ nodes with Amazon's density, so $m \approx 2.77 \times 10^9$ edges; 4-byte node ids.

MEMORY

≈ 100 GB

Graph (CSR) 30 + stubs 22 + duplicate check 44 + per-node counts 4 GB, at the peak.

Preliminary: our prototype uses ≈ 80 B/edge → ≈ 220 GB unless optimised.

RUNTIME

≈ 15 min

Random access dominates: 8.3×10^9 accesses × 100 ns on one core; 64 cores give at most 15× (Amdahl).

Preliminary: 10^7 nodes in 30 s on one core.

PRICE

≈ €11 700

128 GB RAM €4 480 + 64 cores €7 040 + 1 TB SSD €160; with 256 GB ≈ €16 200.

Or free: one SCC node (2.3 TB RAM) holds it 10–20×. Scale up, not out.

Questions?

TARGET

Amazon’s triangles with exact degrees

PREDICTION

Basic model: at most ≈ 0.077 of 0.205, gap explained in advance

COST AT 10^9 NODES

≈ 100 GB, ≈ 15 min, ≈ €11 700